Multi Hypothesis Tracker based on Tracklets (MHTT)

Alexandre EDELINE Théophane BOURDON

March 30th, 2026

Contents

1 Introduction **2**
1.1 Purpose of the tracker . . . 2
1.2 What is a tracklets based tracker ? . . . 2
1.3 The multi-hypothesis paradigm . . . 3
1.4 Algorithm summary . . . 8

2 The scoring process **10**
2.1 Scoring the node . . . 10
2.2 Scoring the branches . . . 12

3 Selecting the best hypothesis **13**
3.1 Building the incompatibility graph . . . 13
3.2 The Maximum Weighted Independent Set (MWIS) problem . . . 14

4 Lifecycles **15**
4.1 Lifecycle of the trees . . . 15
4.2 Lifecycle of the tracks . . . 15
4.3 Lifecycle of the tracklets . . . 16

5 Details on the Kalman filter **17**
5.1 The Kalman filter . . . 17
5.2 Comparison with other filters . . . 19

6 Details on the embeddings regressors **21**

7 Training the hyperparameters **23**

1 Introduction

1.1 Purpose of the tracker

This algorithm is an online, multi-target, human tracker based on computer vision detections. It aims to build multiple tracks from a set of atomic detections. Namely a detection refers to the information that someone was at a specific point at a specific time and a track should be a succession of such detections referring to a unique person. The algorithm is said *online* because it builds tracks at the pace detections arrive, taking fresh detections to expand old tracks or to start new ones.

1.2 What is a tracklets based tracker ?

The tracker is tracklets-based. Let's explain what this means.

At each timestep τ, running the detector on the corresponding video frame generates a batch of detections: $\mathcal{B}_\tau = \{d_\tau^1, \ldots, d_\tau^{n_\tau}\}$. A traditional tracker would directly attempt to extend existing tracks using these detections as they arrive.

Our tracklets-based tracker works differently: it waits over several timesteps $\tau_0, \tau_1, \ldots, \tau_k$ and first links the detections from batches $\mathcal{B}_{\tau_0}, \mathcal{B}_{\tau_1}, \ldots, \mathcal{B}_{\tau_k}$ together to build short, high-confidence track fragments called tracklets. Only then does the tracker extend existing tracks—not from raw detections, but from these pre-built tracklets (see Fig. 1).

Why use tracklets? Using tracklets presents various benefits that compensate for the added complexity of this preprocessing step:

1. **Extended time horizon.** It increases the time horizon over which the tracker makes its decisions. This will become clearer when presenting the multi-hypothesis paradigm, but consider the following example: if the tracker does not use tracklets and works over a horizon of three batches of detections at 5 Hz, it means a decision window of $3 \times 1/5 = 0.6$ s. Using tracklets, if the tracker works over three batches of tracklets, assuming four-detection-long tracklets at 5 Hz, it yields a decision window of $4 \times 1/5 \times 3 = 2.4$ s. Since the multi-hypothesis paradigm uses trees that grow by one stage per batch, making decisions over 2.4 s at 5 Hz in a non-tracklet-based context would require trees of depth $2.4 \times 5 = 12$, while with tracklets of 4 detections it only requires a depth of $2.4 \times 5/4 = 3$. As tree complexity grows exponentially with depth, this represents a clear computational advantage for an online tracker.

2. **Richer information.** A tracklet is a *succession* of detections, so it carries not only the individual detection data but also dynamic information (velocity, acceleration) that can be exploited by a Kalman filter.

3. **Multi-scale association.** It allows exploiting different linking methods at different stages: short-range association during tracklet construction ($\sim$0.1 s to 1 s), and medium-range association when linking tracklets to tracks ($\sim$1 s to 4 s). This enables using specialized methods adapted to each time scale.

How are tracklets built? The tracklet construction step will be described later… (TODO)

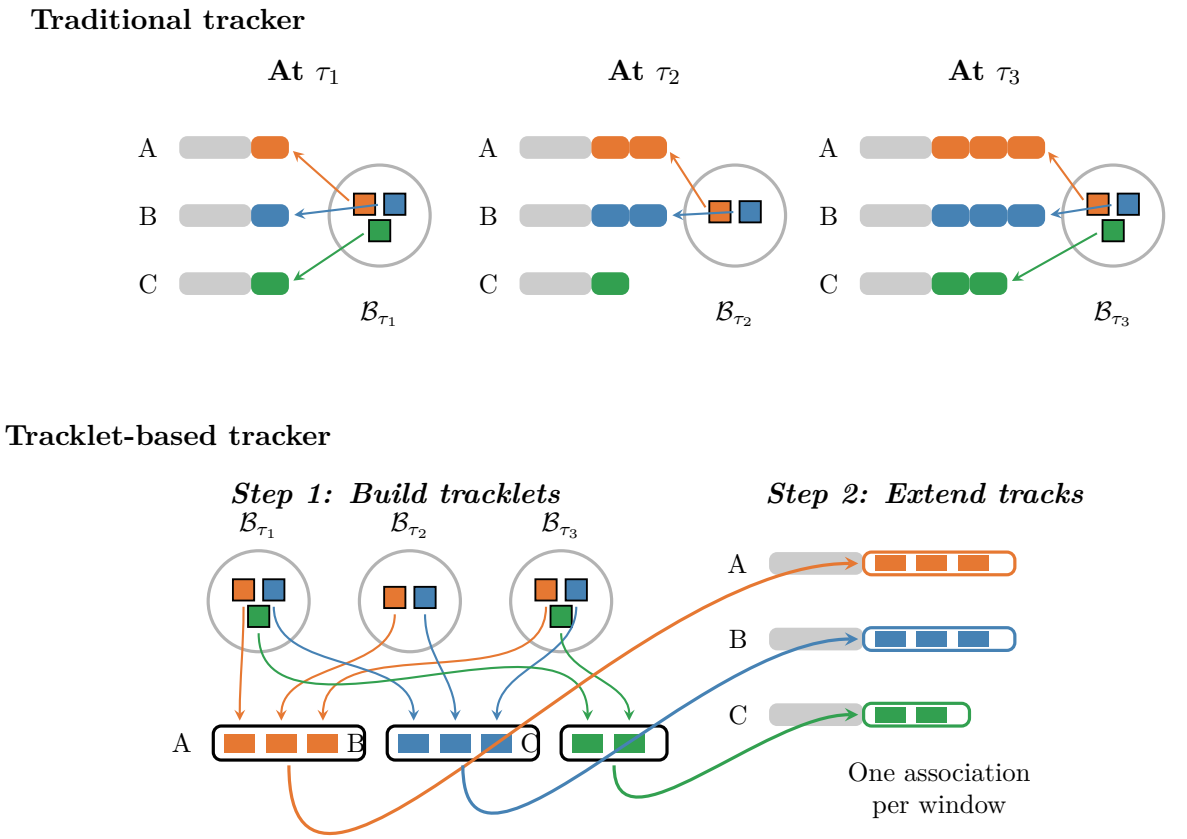

Figure 1: Comparison between traditional tracking (top) and tracklet-based tracking (bottom). Traditional trackers associate detections to tracks at each timestep τ. Tracklet-based trackers first group detections into tracklets, then extend tracks with entire tracklets.

1.3 The multi-hypothesis paradigm

A usual online tracking method would take the tracklets as they come and bind them directly to existing tracks (see Fig. 2).

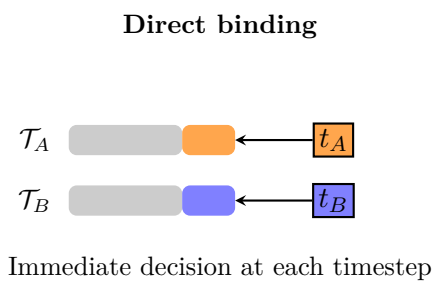

Figure 2: Direct binding commits immediately to an assignment: each incoming tracklet is assigned to one track.

The problem with direct binding is that it forces the algorithm to make permanent decisions at each tracklet batch about where each tracklet is most likely to belong. The multi-hypothesis

paradigm generates hypotheses that allow testing various assignments over a time window of a few tracklets before making a definitive decision. Let's illustrate this with a small example.

Toy example. Assume at time τ_0 we are tracking two persons corresponding to tracks $\mathcal{T}_A$ and $\mathcal{T}_B$. At time τ_1, we receive a batch of two tracklets: $\{t_{1A}, t_{1B}\}$. Two hypotheses can then be formulated for each track: track $\mathcal{T}_A$ can be continued with t_{1A} or with t_{1B}, and the same holds for $\mathcal{T}_B$. This can be modeled through hypothesis trees, to which *dummy nodes* ($\varnothing$) are added to represent the hypothesis that this track is not continued by any tracklet from this batch (the person may be temporarily occluded, or the corresponding tracklet may belong to a new track not yet created). See Fig. 3.

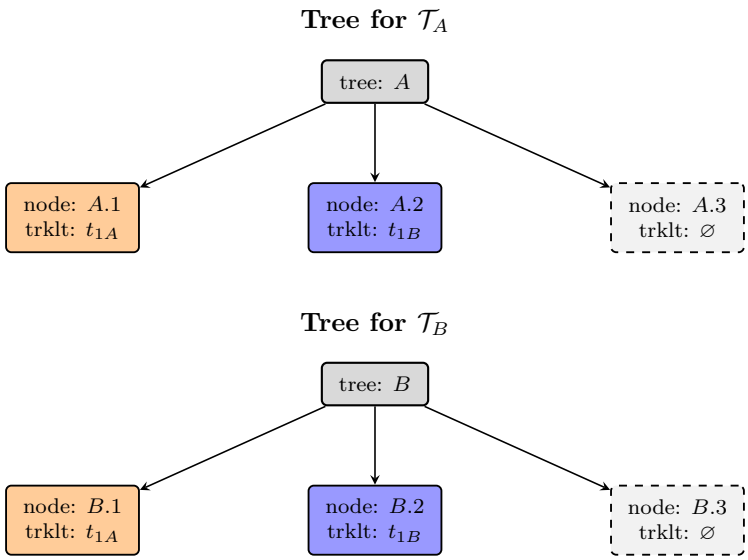

Figure 3: Hypothesis trees after τ_1: each tree has 3 leaves (hypotheses). Each leaf represents a hypothesis about which tracklet continues the track (or $\varnothing$ for none).

Then at a later time τ_2, a new batch of two tracklets is received: $\{t_{2A}, t_{2B}\}$. The trees are extended with these new tracklets to continue formulating hypotheses (see Fig. 4).

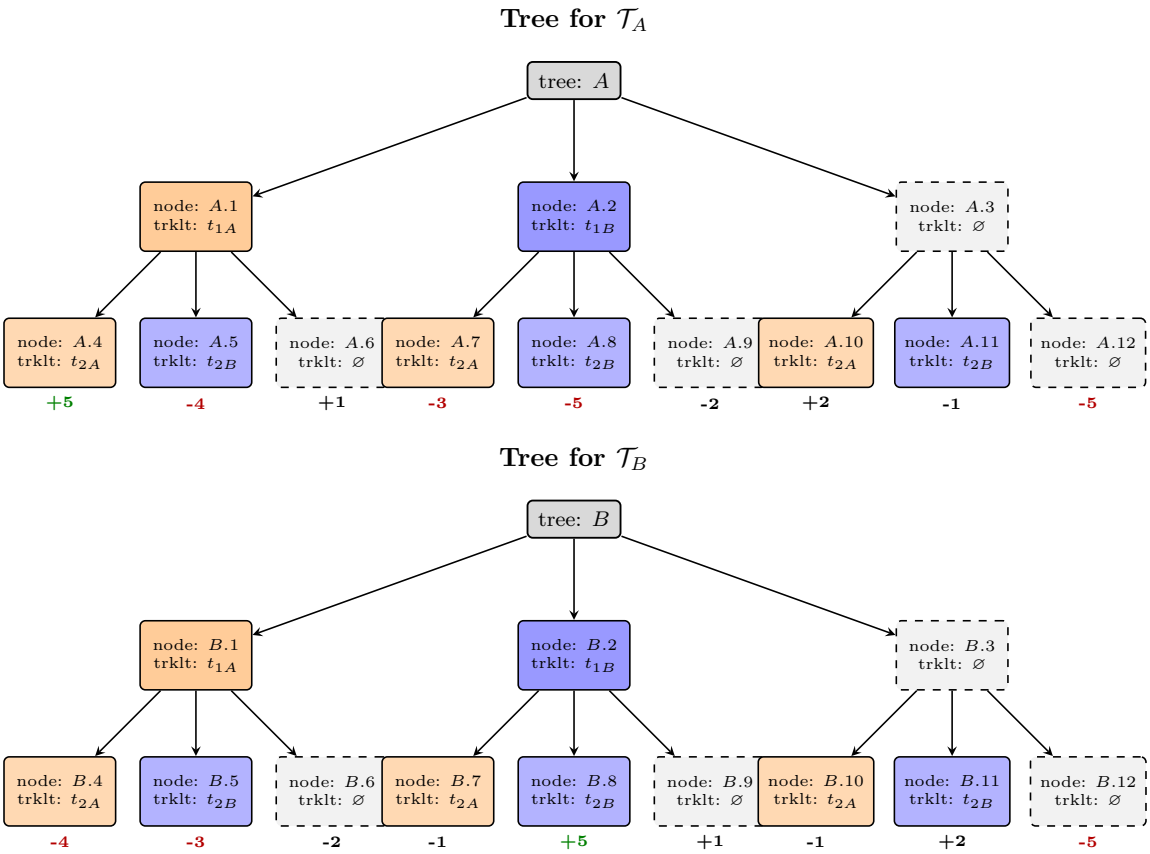

Figure 4: Hypothesis trees after τ_2 with scores. Best hypothesis for $\mathcal{T}_A$: node $A.4$ (path $t_{1A} \to t_{2A}$, score $+5$). Best hypothesis for $\mathcal{T}_B$: node $B.8$ (path $t_{1B} \to t_{2B}$, score $+5$).

Assume at this point we want to choose a definitive hypothesis. We can score each leaf (see Section 2), where a leaf corresponds to a hypothesis. In this case we have $2 \times 3 \times 3 = 18$ hypotheses (2 trees, 9 leaves each). Obviously, we cannot select all hypotheses together because many of them are incompatible with each other. For instance, hypotheses $A.4$ and $B.4$ are incompatible: $A.4$ claims that track $\mathcal{T}_A$ is continued by $t_{1A} \to t_{2A}$, while $B.4$ claims that track $\mathcal{T}_B$ is also continued by $t_{1A} \to t_{2A}$. Since a tracklet can only belong to one track, these two hypotheses cannot both be true.

The resolution will be detailed in Section 3, but the core idea is to build a weighted graph with all leaves as vertices (using hypothesis scores as weights), add edges between incompatible hypotheses (those sharing a tracklet), and then find the subgraph that maximizes total weight while ensuring that no two selected vertices are connected (see Fig. 5). This problem is known as the *Maximum Weighted Independent Set* (MWIS) [**?**]. **It has the significant drawback of being NP-hard, so it is important to keep the total number of leaves as small as possible.**

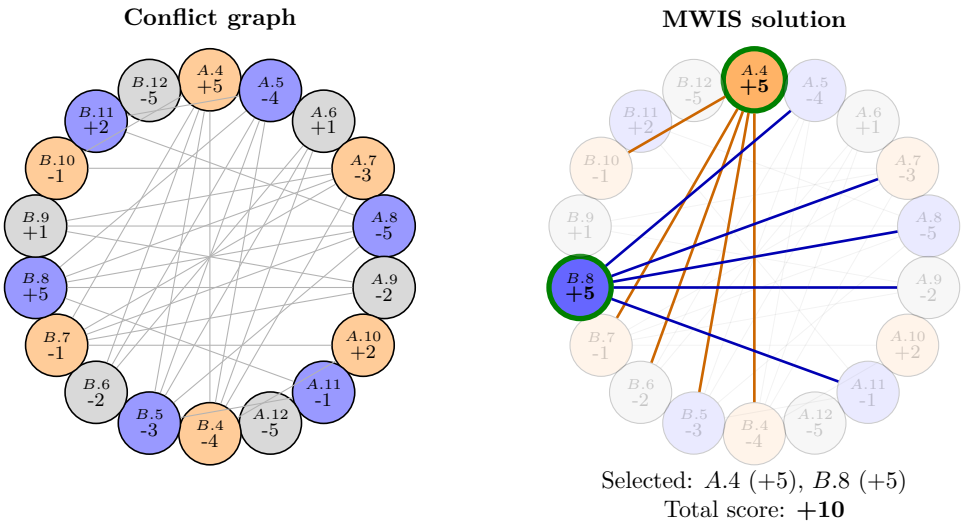

Figure 5: Left: conflict graph where each vertex is a hypothesis and edges connect incompatible hypotheses (those sharing a tracklet). Right: MWIS solution selecting $A.4$ and $B.8$ (no edge between them), maximizing total score.

Tree pruning. Once the MWIS solver has selected a set of compatible hypotheses (here $A.4$ and $B.8$), the trees must be pruned to keep their size bounded. Without pruning, trees would grow indefinitely as new tracklet batches arrive, making the MWIS problem intractable.

The pruning mechanism works as follows. Given a scan depth N_{scan}, we climb up the selected branch from the leaf by N_{scan} stages. All nodes above this point are removed, and the node at this depth becomes the new root. The tracklets corresponding to the pruned portion of the selected branch are *committed* to the track (they are no longer hypothetical). All branches that do not pass through this new root are discarded.

In our example with $N_{\text{scan}} = 1$ (see Fig. 6):

- For tree A: the selected leaf is $A.4$ (path: root $\rightarrow A.1 \rightarrow A.4$). Climbing up 1 stage from $A.4$ leads to $A.1$, which becomes the new root. The tracklet t_{1A} (from the original root to $A.1$) is committed to track $\mathcal{T}_A$.
- For tree B: the selected leaf is $B.8$ (path: root $\rightarrow B.2 \rightarrow B.8$). Climbing up 1 stage from $B.8$ leads to $B.2$, which becomes the new root. The tracklet t_{1B} (from the original root to $B.2$) is committed to track $\mathcal{T}_B$.

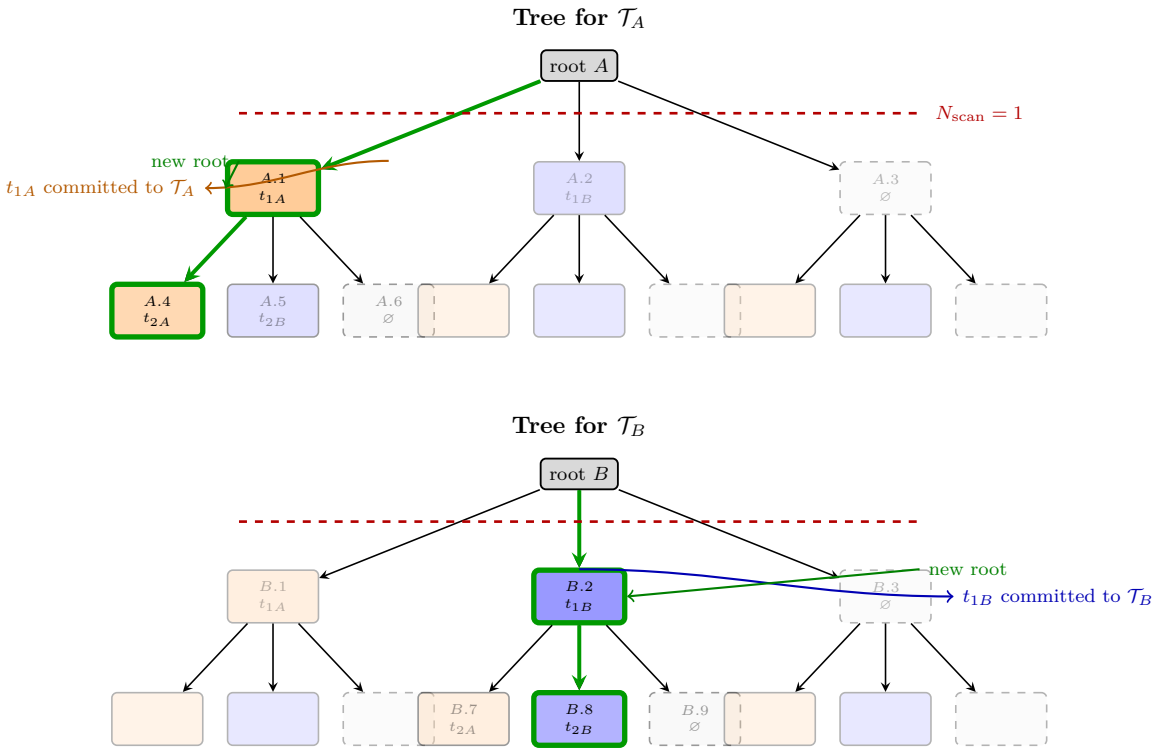

Figure 6: Trees before pruning with $N_{\text{scan}} = 1$. The selected branches (green borders) show the path from root to the chosen hypotheses ($A.4$ and $B.8$). The dashed red line indicates the pruning depth. Faded branches will be discarded. The nodes $A.1$ and $B.2$ become the new roots.

After pruning, the trees are reduced to a single level each, ready to be extended again when the next batch of tracklets arrives (see Fig. 7).

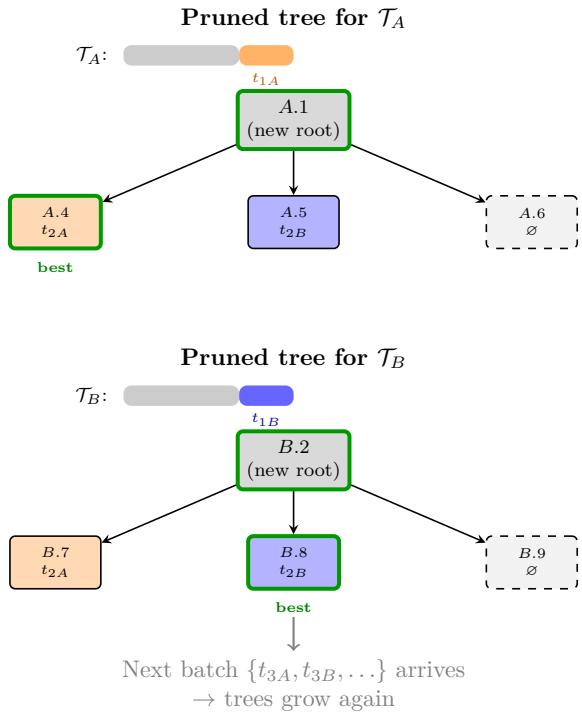

Figure 7: Trees after pruning. Each tree is reduced to depth 1. The tracklets t_{1A} and t_{1B} are now committed to their respective tracks (shown in progress bars). The best current hypotheses are marked. When a new tracklet batch arrives, each leaf will spawn new children, and the cycle repeats.

1.4 Algorithm summary

Algorithm summary. The complete multi-hypothesis tracking algorithm can be summarized as follows. At regular intervals, the tracker receives a new batch of tracklets from the tracklet builder. The algorithm then decides whether to prune the trees (if the number of hypotheses has grown too large) before extending all trees with the new tracklets.

A key aspect not detailed in the toy example above is the creation of new trees. For each incoming tracklet, a new tree is created with that tracklet as its root. This accounts for the possibility that the tracklet corresponds to the first appearance of a new individual. If such a tree is later selected by the MWIS solver, it initiates a new track. In our implementation, each tree is identified by the ID of the tracklet that first grounded it.

Another important detail concerns the pruning conditions. A tree is only pruned if its depth exceeds a maximum depth $D_{\max}$. Trees that have not yet reached this depth are left untouched. Furthermore, if a tree reaches $D_{\max}$ but none of its leaves are selected by the MWIS solution, the tree is simply deleted (the corresponding track hypothesis is abandoned).

Algorithm 1: Multi-Hypothesis Tracklet Tracker (MHTT)

Input: Stream of tracklet batches $\{\mathcal{B}_1, \mathcal{B}_2, \ldots\}$, max leaves $N_{\max}$, max depth $D_{\max}$, scan depth N_{scan}

Output: Set of confirmed tracks $\{\mathcal{T}_1, \mathcal{T}_2, \ldots\}$

```
F ← ∅ ;                                          ▷ Forest of hypothesis trees
for each incoming tracklet batch B_τ do
    ▷ Step 1: Check if pruning is needed
    N_leaves ← total number of leaves in F;
    if N_leaves ≥ N_max then
        Score all leaves in F;
        Build conflict graph G from leaves;
        H* ← SolveMWIS(G) ;                      ▷ Select best compatible hypotheses
        for each tree T ∈ F do
            if depth(T) < D_max then
                Skip pruning for T ;             ▷ Tree not mature enough
            else if no leaf of T is in H* then
                Delete T from F ;                ▷ Abandon this track hypothesis
            else
                h* ← selected leaf of T in H*;
                Climb up N_scan stages from h* to find new root;
                Commit pruned tracklets to track identified by T's root ID;
                Discard all branches not passing through new root;

    ▷ Step 2: Create new trees for potential new individuals
    for each tracklet t ∈ B_τ do
        Create a new tree T_t rooted at t (identified by t's ID);
        Add T_t to F;

    ▷ Step 3: Extend existing trees with new tracklets
    for each tree T ∈ F do
        for each leaf ℓ in T do
            for each tracklet t ∈ B_τ do
                Add child node for t under ℓ;
            Add dummy child node (∅) under ℓ;
```

This algorithm maintains a balance between exploration (keeping multiple hypotheses alive) and computational tractability (pruning when the number of hypotheses exceeds $N_{\max}$). The key parameters are:

- $N_{\max}$: controls the maximum problem size for the MWIS solver.
- $D_{\max}$: minimum tree depth before pruning is allowed (trees need sufficient evidence before committing).
- N_{scan}: controls how much of the tree is committed at each pruning step (higher values commit more tracklets but reduce flexibility).

2 The scoring process

To find the best hypothesis set, each hypothesis (that is, each branch of each tree) is scored. To do this each node of each tree is scored and then for each hypothesis, scores from all nodes on the branch are aggregated to score the hypothesis.

2.1 Scoring the node

Symbol	Name	Description
w_d	`dynamic_score_weight`	Weight for dynamic score vs appearance score ($w_d \in [0, 1]$)
G_d	`dynamic_gate_threshold`	Minimum dynamic score to create a node
G_a	`appearance_gate_threshold`	Minimum appearance score to create a node

Table 1: Hyperparameters for node scoring

Scoring a node means scoring a tracklet *in the context of a hypothesis*, it aims to quantitatively answer the question *how well does this tracklet fit into the hypothesis.* The node scoring relies on two main components: the dynamic likelihood and the appearance similarity. Note also that for computing efficiency if a node score is particularly bad, this node is not even added to the tree, it is said to be gated.

Node scoring a dynamic score S_d (also called motion score) is built and so is an appearance score S_a. The final score is simply a combination of both : $S = w_d \times S_d + (1 - w_d) \times S_a$, where $w_d \in [0, 1]$ is the `dynamic_score_weight`. Also if $S_d < G_d$ or $S_a < G_a$, the node is not even created. G_d being the `dynamic_gate_threshold` and G_a the `appearance_gate_threshold`.

The dynamic score. To evaluate the dynamic likelihood for a tracklet to fit into a hypothesis, we run a Kalman filter estimation and we evaluate the Mahalanobis norm of the innovation of measurement coming from the tracklet data. When a new node is added to continue a hypothesis, this hypothesis has a Kalman state x_{τ_0}, the tracklet of the new node carries n detections, so it gives n new measurements: $y_{\tau_1}, \ldots, y_{\tau_n}$. The new Kalman states are computed from those measurements, at each iteration an innovation (surprise) term is computed along: $\underline{s}_{\tau_k}$ for $k \in \{1, \ldots, n\}$ as an innovation covariance matrix $\underline{\underline{S}}_{\tau_k}$. This leads to a rigorous estimation of the log-likelihood ratio against a null hypothesis (for Gaussian noise):

$$L_{\tau_k} = -0.5\underline{s}_k^T \cdot \underline{\underline{S}}_k \cdot \underline{s}_k - \ln |\underline{\underline{S}}_k| - 0.5D_{mes} \ln 2\pi + 0.5D_{mes} \ln N_{pix} \tag{1}$$

where D_{mes} is the dimension of the measurement space (in our case $D_{mes} = 8$, see Section 5) and N_{pix} is the number of pixels on the image, so $N_{pix}^{D_{mes}}$ is the size of the measurement space.

The motion score might be simply taken as :

$$L = \sum_k L_{\tau_k} \tag{2}$$

However there is empirical evidence that it's not the best choice, because if the tracklet is long enough, even if the motion is not likely in the first frames τ_1, τ_2... (that is, L_{τ_1}, L_{τ_2}... are low) the filter will adapt and eventually will find the values of the last log-likelihood ratios *not that bad* (that is $...L_{\tau_{n-1}}, L_{\tau_n}$... not so low). Such a phenomenon would smooth the dynamic score and so mitigate the signal that there is a really bad likelihood at the tracklet's junction. That is why it is preferred to use

$$L = \sum_{k=1}^{n} w_k L_{\tau_k} \tag{3}$$

with the w_k choosen to give more weight to the first values, for now we use $w_k = \frac{k^{-1}}{\sum_{j=1}^{n} j^{-1}}$ that decrease and sum up to one regardless the tracklet's length. **[?] Maybe we should choose decreasing weights with a tunable parameter ?**

The appearance scoring To score the appearance of a tracklet, a regressor $\mathcal{R}_T$ is trained online for each tree T (that is for each potential track). So that given an embedding vector $\underline{e}$, $\mathcal{R}_T(e)$ should be close to 1 if the vector is similar to the ones of the tree and close to 0 otherwise. More details on how this regressor works are given in Section 6. Namely it is trained to *discriminate* the embedding vectors of its tree and discard the other ones thanks to the embedding covariance information.
Inside a hypothesis tree T, the appearance of a tracklet t holding the embeddings $\underline{e}_1, \ldots, \underline{e}_n$ is scored as follows.

Step 1: Compute regressor outputs. For each embedding $\underline{e}_i$ and each tree T_j in the forest (with N_T trees total), we compute the raw regressor output $r_{ij} = \mathcal{R}_{T_j}(\underline{e}_i)$.

Step 2: Softmax normalization. The regressor outputs are converted to probabilities via a softmax over all trees:

$$p_{ij} = \frac{\exp(r_{ij})}{\sum_{k=1}^{N_T} \exp(r_{ik})} \tag{4}$$

This gives, for each embedding $\underline{e}_i$, the probability p_{ij} that it belongs to tree T_j.

Step 3: Log-likelihood ratio against uniform null hypothesis. The null hypothesis assumes the embedding could belong to any tree with equal probability $\frac{1}{N_T}$. For tree T (with index j^*), the log-likelihood ratio for embedding $\underline{e}_i$ is:

$$\ell_i^{(T)} = \ln\left(\frac{p_{ij^*}}{1/N_T}\right) = \ln(p_{ij^*}) + \ln(N_T) \tag{5}$$

Step 4: Aggregate over the tracklet. The appearance score for the tracklet t in tree T is the mean LLR over all its embeddings:

$$S_a^{(T)}(t) = \frac{1}{n} \sum_{i=1}^{n} \ell_i^{(T)} \tag{6}$$

This scoring mechanism has a key property: it is *competitive* across trees. A high score for tree T necessarily implies lower scores for other trees, which helps the assignment algorithm distinguish between hypotheses.

[?] Pour l'instant j'utilise pas la couleur du torse - je m'en sers dans la construction des tracklets mais pas ici, c'est surement une piste d'amélioration

Scoring the dummies.

Scoring the grounding nodes.

Symbol	Name	Description
α	`young_penalty_strength`	Penalty magnitude for newly created tracks
β	`maturity_rate`	Rate at which the penalty decays (higher = faster maturity)

Table 2: Hyperparameters for branch scoring

2.2 Scoring the branches

Scoring the hypothesis branches wisely may be challenging.
A naïve approach would be to take the average nodes' scores composing the branch. However it would discard information about the hypothesis' length. For instance: a branch of length four would not have any advantage over a branch of length two due to the averaging.
Another idea would be to sum the scores instead of averaging them. Again it would partially take account of the length, yes if we have a branch of length four with all nodes scoring to one its score would be of four and a branch of length two with also its nodes scoring to one would score two, but let's consider a more tricky case. Say we have two branches of length three with whose nodes' score are similar. Say each node scores to one. This would lead each branch to an hypothesis score of three. Now assume that the first branch belongs to a tree corresponding to a track that is 15s old while the second branch is part of a tree that has not even started a track yet. One would clearly prefer the first hypothesis over the second one, so the scores should reflect this.
With this preliminary examples, the reader should understand why we choose to score the hypothesis with :

$$\mathcal{S} = \exp\left(\underbrace{\frac{1}{l}\sum_{i=1}^{l} s_i}_{\text{local coherence}} - \underbrace{\alpha e^{-\beta\,\Delta t}}_{\text{penalty for too young hypothesis}} \right) \tag{7}$$

where l is the length of the branch, s_i are the nodes' scores, α and β are two tunable parameters and Δt is the time elapsed since the creation of the tree. The exponential ensures positivity of the score since the s_i can be negative. The average score term only takes account of the nodes that are still in the tree (i.e., the branch nodes, not the committed ones), so it evaluates the current coherence of the hypothesis. The second term adds a penalty to really young tracks (note that this penalty goes to zero for hypothesis continuing mature tracks), as illustrated in Fig. 8. The parameter α reflects *how strong a new hypothesis is penalized* while β reflects *how fast a track in considered as mature.* **[?] Maybe we could find a unique parameter ?**

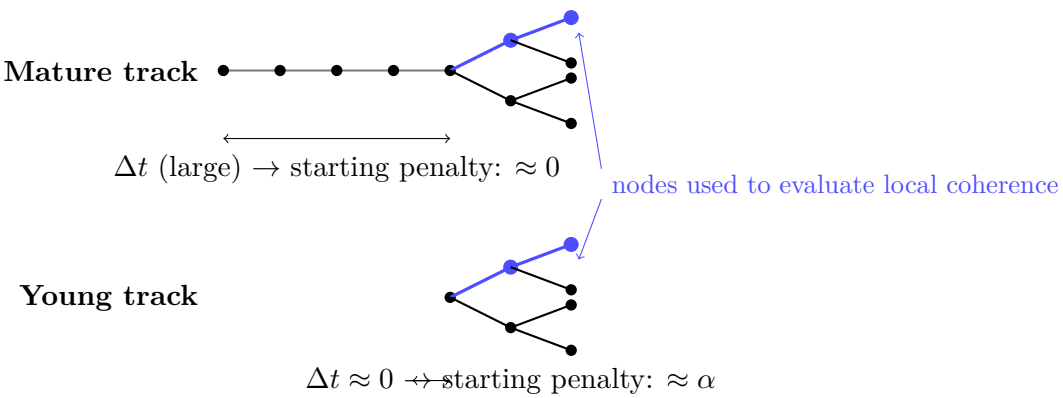

Figure 8: Comparison of hypothesis scoring between a mature track (top) and a young track (bottom). Both highlighted branches have identical node scores, yet the mature track benefits from a near-zero penalty while the young track incurs a penalty close to α.

3 Selecting the best hypothesis

Choosing the best hypothesis among the formulated ones is done by solving a graph optimization problem. It comes down to a linear integer programming problem (LIP): the Maximum Weighted Independent Set (MWIS) problem. The objective is to select weighted vertices over an incompatibility graph (the vertices being the branch hypotheses and their weight being their score) so it maximizes the total weight, while ensuring none of the selected vertices are connected (the edges representing the fact that both extremities are hypotheses sharing at least one tracklet, so are incompatible).

3.1 Building the incompatibility graph

Before formulating and solving the MWIS problem, the undirected weighted conflict graph $G = (V, E, w)$ must be constructed. The vertices represent hypotheses (leaves of the trees), weighted by their branch scores. The edges connect incompatible hypotheses (those that share at least one tracklet along their branches).

Algorithm 2: Build Conflict Graph

Input: Forest of hypothesis trees $\mathcal{F}$
Output: Conflict graph $G = (V, E, w)$

```
▷ Step 1: Build the vertices
V ← ∅;
w ← ∅ ;                                          ▷ Weight function
for each tree T ∈ F do
    for each leaf ℓ ∈ leaves(T) do
        v ← new vertex for ℓ;
        V ← V ∪ {v};
        w(v) ← BranchScore(ℓ) ;                  ▷ Score of the branch from root to ℓ

▷ Step 2: Build the edges (incompatibility constraints)
E ← ∅;
for each pair of vertices (v1, v2) ∈ V × V with v1 ≠ v2 do
    P1 ← Tracklets(v1) ;                         ▷ Tracklets along v1's branch
    P2 ← Tracklets(v2) ;                         ▷ Tracklets along v2's branch
    if P1 ∩ P2 ≠ ∅ then
        E ← E ∪ {(v1, v2)} ;                     ▷ Hypotheses are incompatible
return G = (V, E, w)
```

Complexity considerations. Let N be the total number of leaves across all trees. Step 1 runs in $O(N \cdot D)$ where D is the maximum tree depth (to compute branch scores). Step 2 naively requires $O(N^2 \cdot D)$ comparisons. In practice, this can be optimized by indexing vertices by the tracklets they contain, reducing the edge-building step to iterating only over pairs that share at least one tracklet.

3.2 The Maximum Weighted Independent Set (MWIS) problem

The MWIS on a graph $G = (V, E)$ with weights $(w_v)_{v \in V}$ is to find a binary value for the variables $(e_v)_{v \in V}$, where $e_v = 1$ means the node v is selected (the hypothesis corresponding to v is selected as a *best hypothesis*) and $e_v = 0$ means the node v is discarded (the hypothesis branch corresponding to v is not selected now). It can be formulated as follow :

$$\max_{(e_v)_{v \in V}} \sum_{v \in V} e_v w_v, \tag{8}$$

$$\forall (v_1, v_2) \in E : e_{v_1} + e_{v_2} \leq 1. \tag{9}$$

It can be solved using a LIP solver, we use the free and open-source HiGHS solver.

4 Lifecycles

4.1 Lifecycle of the trees

Symbol	Name	Description
$D_{\max}$	`max_depth`	Maximum tree depth before pruning is triggered
D_{scan}	`scan_depth`	Number of stages to preserve below the new root during pruning

Table 3: Hyperparameters for tree lifecycle management

The tracker keeps a *forest of trees* that is all the hypothesis trees available.

Trees creation Every time the tracker receives a new tracklet t, it grounds a tree from this tracklet, it means that a new tree is created with its root referring to the tracklet t.

Trees expansion For each tracklet freshly received, each tree is expanded with a node corresponding to this tracklet through the scoring mechanism described in Section 2.

Internal trees cleaning At each iteration the trees are maintained:

1. The algorithm checks if the trees contain tracklets that have been committed somewhere else, if so the branches containing such tracklets are pruned from the tree (as those hypothesis branches are no longer an option).
2. When the tree reaches a depth of $D_{\max}$ and one of its branches has been selected, the tracklets corresponding to the nodes belonging to the selected branch and located between depth 1 and $D_{\max} - D_{\text{scan}} - 1$ are committed to the tree's track. The node of the selected branch located at depth $D_{\max} - D_{\text{scan}}$ becomes the new root and the subtree starting from it becomes the new version of the tree.

Trees deletion A tree is deleted in three cases:

1. when the tracklet held by its root has been committed elsewhere.
2. when its depth surpasses $D_{\max}$ and none of its branches has been selected.
3. when its associated track is terminated for any reason.

When deleted, if the tree is linked to an active track, this track is marked as terminated.

4.2 Lifecycle of the tracks

Tracks' creation When a fresh tree commits some tracklets (during the pruning phase), if no track is attached to this tree, a new track is started with the freshly committed tracklets. The track holds the committed tracklets while the tree can be seen as all the potential futures of the track.

Tracks' ending A track ends in two situations:

1. When its tree is deleted.
2. When it consecutively receives more than $D_{\max} + 1$ dummy tracklets.

The second condition is necessary otherwise there would be trees and tracks growing endlessly with only dummy tracklets.

4.3 Lifecycle of the tracklets

The tracker stores the set of all available tracklets, they can be `genuine` tracklets or `dummy` tracklets. The tracklet is identified with a unique identifier. Each node of the trees points to a tracklet. When a tree commits a tracklet into the track, the tracklet becomes a `committed` tracklet and signals that this tracklet is no longer available. The available tracklets set is regularly cleaned: the tracklets that are old enough so they don't belong to any tree anymore are purged.

5 Details on the Kalman filter

5.1 The Kalman filter

Symbol	Name	Description
σ_{bbox}	`sigma_bbox`	Measurement noise std for YOLO bounding box center
σ_{head}	`sigma_head`	Measurement noise std for head keypoints
σ_{body}	`sigma_body`	Measurement noise std for body keypoints (torso, neck)
$\sigma_{Q_{\text{bbox}}}$	`sigma_q_bbox`	Process noise std for bbox (CWNA acceleration)
$\sigma_{Q_{\text{shift}}}$	`sigma_q_shift`	Process noise std for keypoint shifts (random walk)

Table 4: Hyperparameters for the Kalman filter

The state space The Kalman filter mixes information from both YOLO and Pose detectors. The filter aims to estimate the state space:

$$\underline{x} = [x_{\text{bbox}}, y_{\text{bbox}}, v_{x\,\text{bbox}}, v_{y\,\text{bbox}}, \delta x_{\text{head}}, \delta y_{\text{head}}, \delta x_{\text{torso}}, \delta y_{\text{torso}}, \delta x_{\text{neck}}, \delta y_{\text{neck}},]^T.$$

Where the subscript `bbox` refers to the center point of the YOLO bounding box, the `head` subscript refers to the barycenter of the head's keypoints from the Pose model, the `torso` subscript refers to the barycenter of the torso's keypoints and the `neck` subscript refers to the midpoint of the two shoulders' keypoints. The `bbox` components are computed as absolute values in the image while for the keypoints (`head, torso, neck`...) it is the *shift* from the YOLO bounding box that is evaluated, this way the information from both detectors is mixed (see Fig. 9).

The model The evolution of the state is modeled with a constant velocity model for the center of the Yolo bbox and static for the keypoints shifts, so the transition matrix is as follows:

$$F = \begin{bmatrix} 1 & 0 & \Delta t & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & \Delta t & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \tag{10}$$

The measurement space is $\underline{z} = [x_{\text{bbox}}, y_{\text{bbox}}, x_{\text{head}}, y_{\text{head}}, x_{\text{torso}}, y_{\text{torso}}, x_{\text{neck}}, y_{\text{neck}}]^T$, where the keypoint positions are absolute coordinates in the image. The observation matrix must recon-

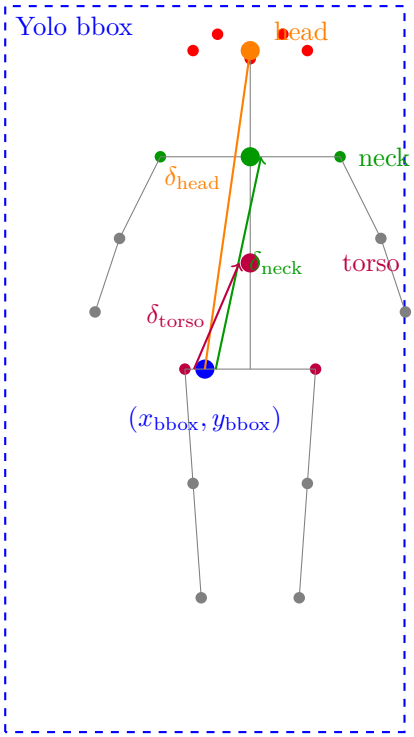

Figure 9: State space representation showing realistic detector misalignment: the Yolo bounding box center $(x_{\text{bbox}}, y_{\text{bbox}})$ and the Pose skeleton do not perfectly align. The δ vectors encode both the body geometry *and* the detector disagreement, allowing the Kalman filter to fuse information from both sources.

struct these absolute positions from the state (bbox center + shifts):

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \tag{11}$$

The noise structure The noise covariances are parameterized by the following standard deviations:

- σ_{bbox}: measurement noise for the Yolo bounding box center
- σ_{head}: measurement noise for the head keypoints (higher uncertainty due to smaller features)
- σ_{body}: measurement noise for the body keypoints (torso and neck)
- $\sigma_{Q_{\text{bbox}}}$: process noise for the bbox acceleration (constant velocity model)
- $\sigma_{Q_{\text{shift}}}$: process noise for the keypoint shifts (random walk model)

Measurement noise covariance R: The measurement noise is modeled as independent Gaussian noise on each measurement component. The head keypoints typically have higher

uncertainty than the body keypoints due to smaller spatial extent.

$$R = \text{diag}\left(\sigma^2_{\text{bbox}}, \sigma^2_{\text{bbox}}, \sigma^2_{\text{head}}, \sigma^2_{\text{head}}, \sigma^2_{\text{body}}, \sigma^2_{\text{body}}, \sigma^2_{\text{body}}, \sigma^2_{\text{body}}\right) \tag{12}$$

Process noise covariance Q: The process noise follows the CWNA (Continuous White Noise Acceleration) model for the bbox and a random walk model for the keypoint shifts.

$$Q = \begin{bmatrix} Q_{\text{bbox}} & 0 \\ 0 & Q_{\text{shift}} \end{bmatrix} \tag{13}$$

where the bbox block uses the standard CWNA formulation:

$$Q_{\text{bbox}} = \sigma^2_{Q_{\text{bbox}}} \begin{bmatrix} \frac{\Delta t^4}{4} & 0 & \frac{\Delta t^3}{2} & 0 \\ 0 & \frac{\Delta t^4}{4} & 0 & \frac{\Delta t^3}{2} \\ \frac{\Delta t^3}{2} & 0 & \Delta t^2 & 0 \\ 0 & \frac{\Delta t^3}{2} & 0 & \Delta t^2 \end{bmatrix} \tag{14}$$

and the shift block models a random walk on the shifts:

$$Q_{\text{shift}} = \sigma^2_{Q_{\text{shift}}} \cdot \frac{\Delta t^4}{4} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \tag{15}$$

5.2 Comparison with other filters

Before choosing this Kalman filter configuration, many state spaces and models have been considered. A benchmark on real data has been conducted to evaluate the ability of different KF configurations to discriminate between a good track continuation and a bad one.

Experiment

Data The data are detections extracted from the detector (mixing YOLO and Pose) from many various videos that were linked together to form various tracks. So the data are various videos v, for each video we have a set of tracks $\mathcal{T}^v = \{T_1, ..., T_n\}$, where each track $T_i = (t^i_1, \dots)$ is a sequence of tracklets.

Benchmark The benchmark is conducted as follows: for each tracklet t^i_k, the Kalman filter is run over three tracklets $t^i_k \to t^i_{k+1} \to t^i_{k+2}$ (it simulates the initialization).

Then we evaluate the log-likelihood ratio (LLR) over a null hypothesis (see Section 2) when the sequence is continued by either: (a) the correct next tracklet t^i_{k+3} (good continuation), or (b) a bad tracklet (the closest in space and time tracklet from a different track T_j, $j \neq i$). This gives two scores: L^+ (good) and L^- (bad). The benchmark evaluates how well the KF discriminates between good and bad continuations: ideally $L^+ > 0$ (the measurement is more likely than the null hypothesis) and $L^- < 0$ (the measurement is less likely than the null hypothesis).

Parameters Optimization To run the benchmark on each KF architecture, the parameters are first tuned using the CMA-ES algorithm and its Python API[1].

[1] CMA-ES: arxiv.org/abs/2402.01373

Findings The key findings are (see Fig. 10):

1. Using the pose estimation is much more efficient than simply the YOLO bbox.
2. At 5Hz using a CV estimation on the c_x, c_y, w, h bounding box space does no better that a simple c_x, c_y one.
3. The head keypoints are by far the most reliable keypoints.

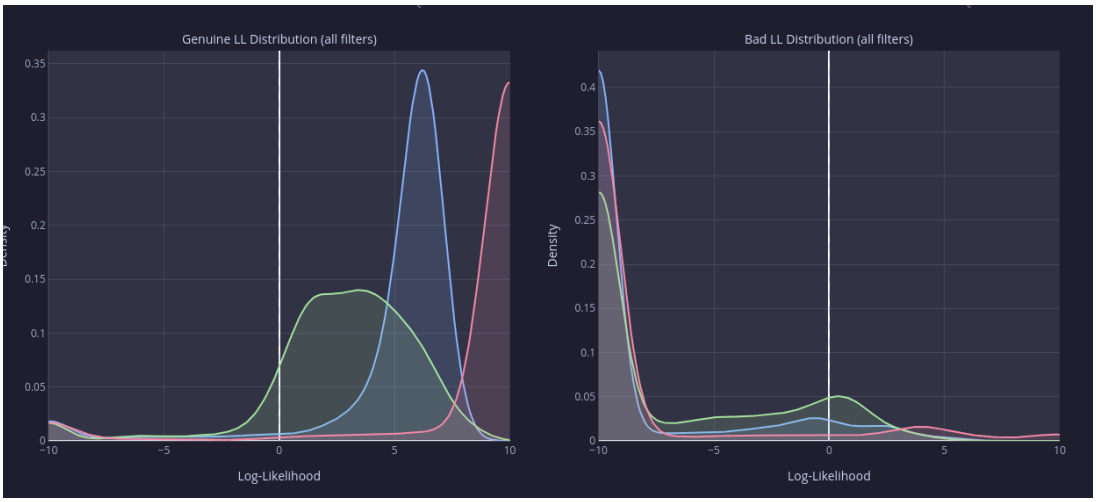

Figure 10: Log-likelihood ratio of the measurement over null hypothesis for good (left) and bad (right) tracklet continuation. The green curve is a CV KF using only the bbox center, the blue one is a CV KF using only the head center, the red curve is the coupled state KF mentioned above.

[?] **J'ai aussi un filtre qui estime le centre de la tête et les biais du torse rapport à la position de la tête, je crois qu'il a d'encore meilleur résultats mais ça me gène un peu de pas du tout utiliser le Yolo conceptuellement.**

6 Details on the embeddings regressors

Symbol	Name	Description
λ	`ridge_regularization`	Regularization parameter for ridge regression
γ	`covariance_momentum`	Momentum for online covariance update ($\gamma \in [0, 1]$)
d	`embedding_dim`	Dimension of the embedding space

Table 5: Hyperparameters for the embeddings regressors

What is a regressor Given a track T, that is a unique persone, assume we have a set $E = \{\underline{e}_1, ..., \underline{e}_n\}$ of n embedding vectors partitioned in two sets E_T, the *true* embeddings (the ones that corresponds to the tracked person) and E_F the *false* emebeddings (the ones that corresponds to someone esle). The goal of the regressor $f_T : E \to [0, 1]$ is to associate a value to one to the vectors $\underline{e} \in E_T$ and close to zero for $\underline{e} \in E_F$. In our case we simply use *linear* regressors, so the regression function f_T can be associated to a vector $\underline{r}_T$ such that : $f_T(e) = \underline{r}_T^T \cdot \underline{e}, \forall \underline{e} \in E$.
Note that the regressor is chosen to disciminate a positive and negative class, which make it much more efficient than a score based on a cosine distance.

How to train a regressor To train the regressor, a *ridge regression* is performed. Given a ground truth vector $\underline{y} \in \mathbb{R}^n$, with $y_i = 1$ if $\underline{e}_i \in E_T$ and $y_i = 0$ of $\underline{e}_i \in E_F$, the ideal regressor $\underline{r}$ would yeild :

$$\underline{r}\underline{\underline{X}} = \underline{y}, \text{with: } \underline{\underline{X}} = [\underline{e}_1, \ldots, \underline{e}_n]. \tag{16}$$

However this problem is ill posed as $\underline{\underline{X}}$ is not even square, so the ridge regression is performed as follow: first, both side of are multiplied by $\underline{\underline{X}}^T$. With d being the dimension of the embedding space, it makes the *covariance* matrix $\underline{\underline{C}} = \underline{\underline{X}} \cdot \underline{\underline{X}}^T \in \mathbb{R}^{d \times d}$ appears:

$$\underline{r}\underline{\underline{C}} = \underline{y} \cdot \underline{\underline{X}}^T. \tag{17}$$

then en ensure inversability and mitigate overfitting, a regularization term is added to $\underline{\underline{C}}$:

$$\underline{r}\left(\lambda \underline{\underline{I}} + \underline{\underline{C}}\right) = \underline{y} \cdot \underline{\underline{X}}^T. \tag{18}$$

So the regressor is given by :

$$\underline{r} = \underline{y} \cdot \underline{\underline{X}}^T \cdot \left(\lambda \underline{\underline{I}} + \underline{\underline{C}}\right)^{-1}. \tag{19}$$

Note that the cost grows as $\mathcal{O}(d^3)$, so is motivates the use of relatively small embedding spaces.

Training an online regressor The previous paragraph explains how to train an *offline* regressor, that is in the case we have fixed the `positive` and `negative` training set. But in out case, the regressors should be retrained online. Indeed every time new tracklets are commited, that is associated to a unique person, the training set E should be completed with those new example.
Hopefully theire is an easy way to update the regressor with new values. Assume we have a set E of n samples and then arrives a new batch E' with n' samples. The new covariance matrix would be $[\underline{\underline{X}}, \underline{\underline{X}}'] \cdot [\underline{\underline{X}}, \underline{\underline{X}}']^T = \underline{\underline{X}}\underline{\underline{X}}^T + \underline{\underline{X}}'\underline{\underline{X}}'^T$. So the new covariance is simply, $\underline{\underline{C}}' = \underline{\underline{C}} + \underline{\underline{X}'X'}^T$. Additionally to give more importance to new information we use a momentum parameter $\gamma \in [0, 1]$ so the final update formula is : $\underline{\underline{C}}' = \gamma \underline{\underline{C}} + (1 - \gamma)\underline{\underline{X}'X'}^T$.

Lifecyle the regressors

Purpose of the regressor In our case, a regressor $\underline{r}_T$ is attached to each tree T. The regressor $\underline{r}_T$ is then used to estimate the likelihood for a tracklet to belongs to T based on its appearance, that is on the tracklet's embeddings vectors (see).

Initialisation of the regressor [?] **Ici j'ai clairement un point dur auquel je ne m'étais pas frotté. C'est assez difficile de maintenir un entrainement cohérent, car a priori tous les régresseurs doivent partager la même matrice de covariance $\underline{\underline{C}}$ et seul les valeurs du vecteur de *ground truth* $\underline{y}_T$ change d'un régresseur à l'autre. C'est d'ailleurs génial il n'y a qu'une inversion pour tous les régresseurs... Le processus actuel est la suivant: j'ajoute à la matrice de covariance les embeddings des tracklets récemment recus via la formule $\underline{\underline{C}} \leftarrow \gamma\underline{\underline{C}} + (1-\gamma)\underline{\underline{C}}'$, pour une track T, dès que je *commit* un tracklet (c'est à dire que la tracklet n'appartient plus à l'arbre mais à la track définitive) elle est marquée comme exemple positif dans le vecteur de vérifté. Mais il reste un point dur, imaginons que tu *commit* une tracklet t à la track T. Avant d'avoir commit t, les embeddings associés à t servaient d'exemple négatifs... et donc parfois ca mène a des résultats pas très bon... L'idée c'est qu'on pourrait entrainer les régresseurs seulement avec les tracklets qui ont été validées, mais ca pose la question de comment on initialise le régresseur à la naissance d'un arbre (parceque pour l'instant c'est facil, on entrainte un nouveau régressseur avec comme exemple positif les embeddings de la tracklet qui fondent l'abre, mais s'il faut attendre que l'abre commit une tracklet avant d'avoir des exemples positif... ca le laisse longtemps non-initialisé...) En fait si on veut initialiser avec la tracklet fondatrice de l'abre les nouveaux régresseurs et ensuite maintenir l'entraintement online, il faudrait une matrice de covariance par nouvel arbre (les arbres matures continueraient a partager la même matrice de covariance mais pour l'initialisation il faudrait une matrice de covariance spécifique a chaque nouvel arbre... c'est pas très élégant. Apparement il y a une formule de "Woodbury" qui pemet d'inverser à mondre cout $\underline{\underline{M}} + \underline{e} \cdot \underline{e}^T$ à partir de $\underline{\underline{M}}^{-1}$ donc ça doit être faisable...)**

7 Training the hyperparameters

Recall that we do have many hyperparameters to tune (see Table 6).

Symbol	Implementation	Description	Trained?
Tree Lifecycle			
$D_{\max}$	`max_depth`	Maximum tree depth before pruning	Fixed
D_{scan}	`scan_depth`	Stages to preserve during pruning	Fixed
Node Scoring			
w_d	`dynamic_score_weight`	Weight for dynamic vs appearance score	Trained
G_d	`dynamic_gate_threshold`	Min dynamic score to create a node	Trained
G_a	`appearance_gate_threshold`	Min appearance score to create a node	Trained
Branch Scoring			
α	`young_penalty_strength`	Penalty magnitude for young tracks	Trained
β	`maturity_rate`	Rate at which penalty decays	Trained
Kalman Filter			
σ_{bbox}	`sigma_bbox`	Measurement noise for YOLO bbox	Trained
σ_{head}	`sigma_head`	Measurement noise for head key-points	Trained
σ_{body}	`sigma_body`	Measurement noise for body key-points	Trained
$\sigma_{Q_{\text{bbox}}}$	`sigma_q_bbox`	Process noise for bbox (CWNA)	Trained
$\sigma_{Q_{\text{shift}}}$	`sigma_q_shift`	Process noise for keypoint shifts	Trained
Embeddings Regressors			
λ	`ridge_regularization`	Ridge regression regularization	Trained
γ	`covariance_momentum`	Momentum for covariance update	Trained
d	`embedding_dim`	Dimension of embedding space	Fixed

Table 6: Summary of all hyperparameters. Fixed parameters are set by design choices (tree depth, embedding model), while Trained parameters are optimized via the training procedure.

To do so we proceed to a two-step training using the CMA-ES *gradient-free optimisation* librairy:

- **Score Training (warmup step):** This first step is not fully aligned with our final desired goal but it is fast, so it should be a good starting point for the final training step. This first training step aims to do a *local* training where we optimise the score generation. For each training sample, a tree trunk is generated with correct tracklets, and is then expanded with 4 leaves: (i) the correct next tracklet t_T, (ii) and (iii) the two nearest in time and space wrong tracklets t_{F1} and t_{F2}, and (iv) a dummy tracklet t_D (see Fig. 11). The loss function is chosen to ensure that we have roughly: $s_T > s_D > s_{F1} \approx s_{F2}$, where s_T is the score of the leaf (i), s_D of the leaf (iv), and s_{F1}, s_{F2} the scores of the leaves (ii) and (iii).

- **End-to-end Training (final training):** The second step is closer to the actual tracker behaviour. The whole tracker is run over the data of various videos, and the loss function rewards long and consistent tracks (without ID switches). The chosen loss is:

$$\mathcal{L}_{\texttt{e2e}} = \sum_{V} \sum_{T \in \texttt{tracks}(V)} \Big[\underbrace{l_T^s}_{\text{length}} \times \big(\underbrace{\gamma_T}_{\substack{\text{consistent}\\ \text{(reward)}}} - p \cdot \underbrace{\iota_T}_{\substack{\text{inconsistent}\\ \text{(penalty)}}} \big) + \underbrace{\tilde{l}_T^s}_{\substack{\text{coherent}\\ \text{length}}} \times \underbrace{\kappa_T}_{\substack{\text{recovered}\\ \text{(bonus)}}} \Big] \tag{20}$$

Track quality indicators:

$\gamma_T = 1$ if track T is consistent

$\iota_T = 1$ if track T has ID switches

$\kappa_T = 1$ if track T recovered[2]

Parameters:

l_T = length of track T

$\tilde{l}_T$ = coherent length of track T

$s \approx 1.5$ (supralinear exponent)

$p \geq 1$ (penalty coefficient)

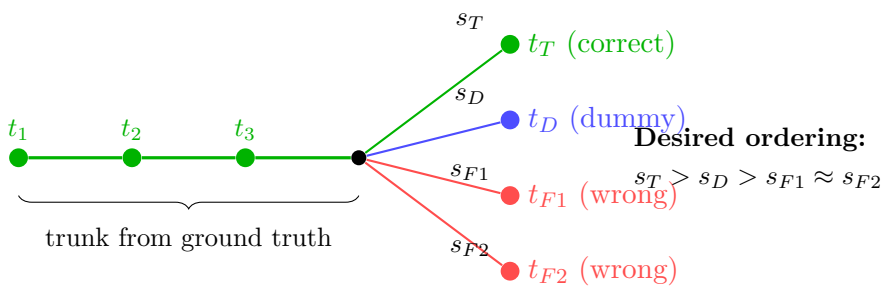

Figure 11: Training configuration for the warmup step. The trunk (green) is built from ground truth data. Four leaves are attached: the correct continuation t_T, two spatiotemporally close but wrong tracklets t_{F1}, t_{F2}, and a dummy tracklet t_D. The loss encourages correct tracklets to score highest, dummy to score intermediate, and wrong tracklets to score lowest.

[2] A track is *recovered* if it had an ID switch but the first and last tracklets share the same identity.

Glossary

detection : information about the presence of an individual at a specific timestep. Those are the input elements of any tracker. There are many detectors in computer vision, but as our trackers rely strongly on the detections they receive, here is a summary of how the detections are built and what information they carry.
A detection comes from the combination of two unitary detectors. The first one is a **YOLO** detector, trained to recognize a *human* class. Given a video frame, for each human presence detected, the YOLO detector yields the coordinates of the corresponding bounding boxes. The second detector is a **multi-pose** detector. Given a video frame, it yields an estimation of the coordinates of various body keypoints for each individual detected. When both detectors find an individual with serious confidence at the same location, the detection is confirmed.
Once the detection is confirmed, it is enriched with appearance information. The first appearance information is an embedding vector resulting from running a feature extractor model on a crop of the YOLO's bounding box. The second appearance information is simply the tone[3] histogram of the person's torso - it roughly answers the question: *What is the color of their shirt?*. This information will be precious to match the detections. . 2

embedding vector : a vector of $\mathbb{R}^d$ that represent the feature extraction result of an image. It is used to estimate the appearance similarity between two detections. . 21

hypothesis tree : a tree whose nodes correspond to tracklets and in which each branch represents a hypothesis on how the root tracklet of the tree can be continued to build a track. Each node has a *score* that reflects how well the corresponding tracklet fits into the hypothesis. Each tree is identified with the ID of the tracklet that first grounded it. . 4, 15

incompatibility graph The incompatibility graph is a weighted, undirected graph where each hypothesis is represented by a vertex whose weight is the score of the hypothesis branch. An edge between two vertices means that the hypotheses corresponding to those vertices are incompatible due to sharing one or more tracklets. . 13

scan depth : given a selected hypothesis leaf, if the tree is of depth D and the scan depth is N_{scan}, the tree is climbed from the leaf N_{scan} stages up to stage $D - N_{scan}$ and the new tree starts from this new root. For example, if the tree is of depth 5 and has a scan depth of 2, the stages of depth 0,1 and 2 will be pruned while the new root will be the 3rd staged node on the selected branch (see Fig. 6).. 6

track : a sequence of detections linked over consecutive timesteps representing a unique identity. As our algorithm is based on tracklets, a track is built as a succession of tracklets. . 2, 3

tracklet : short sequence of detections linked together over a few consecutive frames, representing a partial track. 2, 3

[3] In HSV sense.